ReAct Support Agent on tau2-bench: Scope and Build Document

Oct 12, 2026 · @Arushi

1. Overview and objectives

We will build one ReAct customer-support agent in LangGraph and run it two ways: scored on tau2-bench (retail, airline, telecom) and served as a live demo where an operator signs off every irreversible action. Benchmark credibility and demo quality carry equal weight, so the same graph code powers both.

Objectives

- An agent that reasons, calls one tool, reads the result and decides again across many steps, with guardrails and human approval before WRITE actions.
- A reproducible eval harness reporting task success rate, tool accuracy, escalation rate and pass^1 to pass^4 against the built-in tau2 baseline on the same model.
- A deployed demo on Vercel (Hobby) with an approvals console, three scripted scenarios and full traces in Phoenix.

Proposed success criteria

Area	Criterion	How measured
Capability	pass^1 at or above the tau2 `llm_agent` baseline on the same model, per domain	Full run matrix, 4 trials
Reliability	pass^4 gap to baseline reported per domain, with the guarded config not below the unguarded one	pass^k estimator in `src/evals/metrics.py`
Safety	Write precision higher than the unguarded config; zero unapproved WRITE calls in serve mode	Trajectory analysis, approvals table
Demo	Each of the 3 scenarios runs live in under 5 minutes with a decision timeline and a Phoenix trace deep link	Rehearsal checklist

Area	Criterion	How measured
Operations	Scheduled benchmark run (GitHub Actions or local) completes and publishes to the `/evals` page	GitHub Actions run history

2. Scope

The build covers the text (half-duplex) agent on three tau2 domains, its serving stack and its eval pipeline; voice, knowledge retrieval and model training are out.

In scope	Out of scope
tau2-bench text mode: `retail`, `airline`, `telecom`, `base` task split	`banking_knowledge` domain and voice (full-duplex) mode
LangGraph ReAct graph with bench and serve execution modes	Fine-tuning or RL on the gym interface
tau2 `HalfDuplexAgent` adapter registered as a custom agent	Leaderboard submission (optional, after P5)
FastMCP server exposing the three domain toolkits	Real customer channels (email, WhatsApp, voice)
Guardrails: input, pre-tool, confirmation, domain policy, output	Multi-tenant auth, billing, SSO
Human sign-off: `interrupt()` in serve mode, auto-reviewer modes in bench	Slack or Teams approval routing (later option)
Evals: success rate, tool accuracy, escalation rate, pass^k	Human-graded conversation quality
Phoenix: self-hosted in Docker for benchmark traces, Phoenix Cloud free tier for demo traces, eval scores written back to spans	Arize AX (cloud)
Vercel Hobby (FastAPI and MCP functions), Supabase or Neon free Postgres, GitHub Actions bench runs	GCP and Cloud Run; multi-region, high availability
Approvals console and `/evals` page in EMB brand	Production SLAs

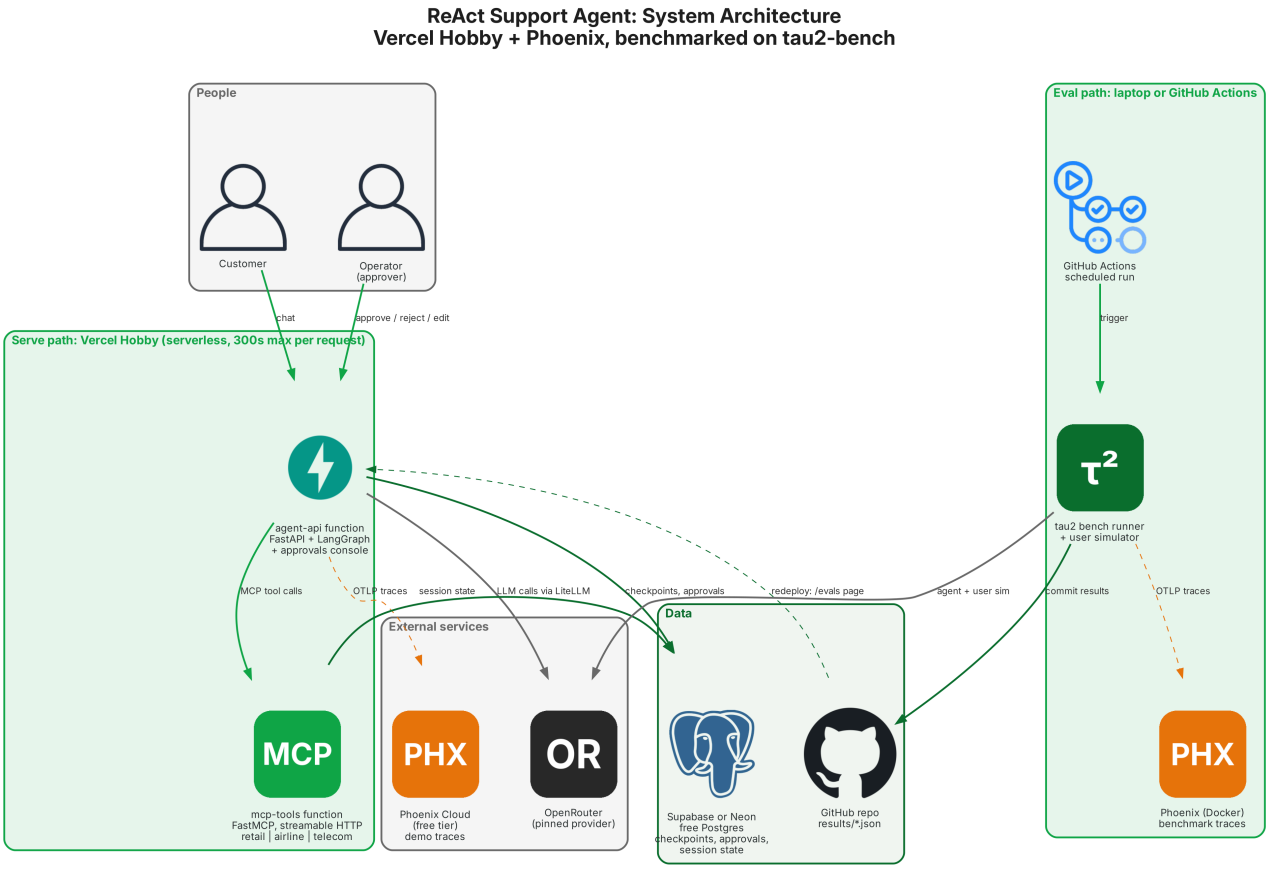
In scope	Out of scope
Docker Compose for local dev (API, MCP, Postgres, Phoenix)	Temporal (needs always-on workers, which Vercel functions cannot run)

Assumptions

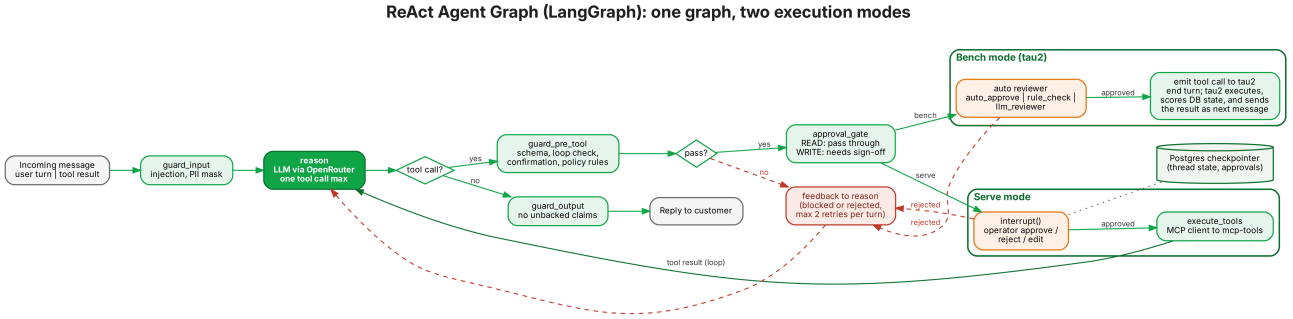
- Pin a tau2-bench release tag; the repo now needs `uv` and Python 3.12 or 3.13.
- All LLM calls go through OpenRouter via LiteLLM, with one pinned provider per model.
- One developer, about 7 weeks including the demo track.
- Vercel Hobby hosts the API and MCP functions for a personal, non-commercial build; it moves to Vercel Pro before any client showing. Supabase or Neon free Postgres is the only hosted datastore (LangGraph checkpoints, approvals, MCP session state).
- The tau2 user simulator model stays fixed for every run so results remain comparable.

3. Architecture

One LangGraph graph serves two paths: a Vercel serve path where customers chat and operators approve, and a laptop or GitHub Actions eval path where tau2 drives the same graph and scores it. Both paths call models through OpenRouter and send traces to Phoenix: Phoenix Cloud for the demo, self-hosted Docker for benchmark runs.



Why two execution modes. tau2's orchestrator executes tools against its own stateful environment and scores the resulting DB state, so in bench mode the graph must hand each tool call back to tau2 rather than run it. In serve mode the graph runs the tool itself through MCP and loops.



Path	Who drives the loop	Tool execution	Sign-off
Serve	LangGraph, per customer turn	`execute_tools` via MCP to `mcp-tools`	Operator via `interrupt()` and the approvals console
Bench	tau2 orchestrator, per turn	tau2 environment, after `emit`	Auto-reviewer set by `approval_mode`

4. Component specifications

Seven components carry the build; only `execute_tools` differs between modes, so every guard and prompt is tested once and used twice.

4.1 LangGraph agent graph (`src/agent/`)

Entry point: `build_graph(mode: Literal["bench", "serve"], tools, policy, llm, approval_mode)` in `src/agent/graph.py`.

State (`src/agent/state.py`):

```
class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
    domain: str
    pending_tool_call: dict | None
    guard_feedback: list[str]      # reasons an action was blocked
    retries_this_turn: int         # capped at 2
    approval: dict | None          # decision, edited_args, reviewer, reason
    step_count: int                # capped at 30 per turn
    decision_note: str | None      # one-line display summary for the
timeline
```

Node	File	Responsibility	Next
`guard_input`	`nodes/guard_input.py`	Injection and role-override patterns, PII masking for traces	`reason`
`reason`	`nodes/reason.py`	LLM with tools bound, `parallel_tool_calls=False`, provider check, decision note	`guard_pre_tool` or `guard_output`
`guard_pre_tool`	`nodes/guard_pre_tool.py`	Schema, loop, confirmation and domain policy checks	`approval_gate` or feedback to `reason`
`approval_gate`	`nodes/approval.py`	READ passes; WRITE needs sign-off (interrupt or auto-reviewer)	`execute_tools`, `emit`, or feedback
`execute_tools`	`nodes/execute.py`	Serve only: MCP `ToolNode` call, result appended	`reason`
`emit`	`nodes/emit.py`	Bench only: return the tool call to tau2 and end the turn	END
`guard_output`	`nodes/guard_output.py`	Block replies claiming actions without a successful tool result	END or `reason`

4.2 tau2 adapter (`src/tau2_adapter/`)

`LangGraphReActAgent` subclasses `LLMConfigMixin` and `HalfDuplexAgent`. It implements `get_init_state` and `generate_next_message`, converts tau2 messages to LangChain messages in `convert.py`, and is exposed through a `create_agent(tools, domain_policy, **kwargs)` factory registered in the forked `src/tau2/registry.py` as `langgraph_react`.

```
def generate_next_message(self, message, state):
    out = self.graph.invoke(
        {"messages": [to_lc(message)]},
        config={"configurable": {"thread_id": state.thread_id}},
    )
    return to_tau2_assistant(out["messages"][-1]), state
```

4.3 MCP tool server (`src/mcp_server/`)

- FastMCP server over streamable HTTP, one process, tools namespaced `retail.*`, `airline.*`, `telecom.*`.
- Each session gets its own copy of the domain DB, stored as JSONB in Postgres keyed by `session_id` (`session_store.py`).

- READ/WRITE classification is derived from the toolkit's tool-type decorators, not a hardcoded list.
- `tests/test_tool_parity.py` asserts the MCP schemas match `env.get_tools()` exactly.

4.4 FastAPI service (`src/api/`)

Method	Path	Purpose
GET	`/domains`	Domains for the operator's selector: key, label, one-line description, sample customer profiles
POST	`/sessions`	Operator only: start a session for the chosen domain (required, locked for the session): `{domain, customer_profile_id}` returns `session_id`
POST	`/sessions/{id}/messages`	Send a customer turn; streams agent output and decision step events (SSE)
GET	`/sessions/{id}`	Transcript, state and Phoenix trace link
GET	`/sessions/{id}/events`	Decision timeline events for replay, in order, each with its Phoenix trace id
GET	`/approvals?status=pending`	Operator queue; optional domain filter
POST	`/approvals/{id}`	`{decision: approve, reject or edit, edited_args, reason}` ; resumes the graph with `Command(resume=...)`
GET	`/evals/latest`	Latest benchmark metrics from the committed results/ folder
GET	`/healthz`	Liveness for Vercel and uptime pings

4.5 Approvals console (`src/api/ui/`)

- Pending queue: tool name, arguments diff, policy-check results, customer confirmation quote and a Phoenix trace link.
- Actions: approve, reject with reason, edit arguments then approve.
- `/evals` page: per-domain success rate, pass^1 to pass^4, tool accuracy and escalation rate for the latest run.
- Styling: EMB green `#12A54A` , Inter, logo header.

4.6 Domain selection by the operator

The operator or admin chooses the domain in the console and starts the session; the customer never picks one and lands straight in a chat locked to that domain.

1. In the console, the operator picks Retail, Airline or Telecom from a selector filled by `GET /domains`, plus an optional sample customer profile for demos.
2. "Start session" calls `POST /sessions` with `{domain, customer_profile_id?}` (operator token required). It creates a fresh copy of that domain's DB and returns `session_id` and a customer chat link `/chat/{session_id}`.
3. The customer opens the link and chats; the header shows a read-only domain badge. The agent loads only that domain's `policy.md` and binds only its `<domain>.*` MCP tools.
4. To change domain, the operator ends the session and starts a new one; the domain never switches mid-conversation.
5. The approvals queue shows each item's domain and can be filtered by it.

Enforcement happens in three places: the API rejects session creation without a valid operator token or with a missing or unknown `domain` (`retail`, `airline`, `telecom` only), `guard_pre_tool` blocks any tool outside `<domain>.*`, and the MCP server checks each call against the session's stored domain. Every demo scenario starts with the operator choosing the domain.

4.7 Decision visibility: timeline and Phoenix

The demo shows every decision two ways: a plain-language Decision timeline beside the chat for business viewers, and the full Phoenix trace one click away for technical viewers.

Decision timeline (panel beside the customer chat and in the console)

Step	Source node	Shown as
Understand	`reason` (decision note)	One-line restatement of what the customer wants
Look up	`execute_tools` (READ)	Tool name and the key fact returned
Check policy	`guard_pre_tool`	A pass or block per rule, with the reason
Propose action	`approval_gate`	The WRITE tool and its arguments
Sign-off	`approval_gate`	Waiting, approved or rejected, with reviewer and reason

Step	Source node	Shown as
Act	`execute_tools` (WRITE)	The result of the action
Escalate	transfer tool	Why the agent handed off to a human
Reply	`guard_output`	The final message, after the output check

- `POST /sessions/{id}/messages` runs the graph with LangGraph `stream_mode="updates"` and emits SSE `step` events: `{seq, type, node, title, detail, verdict, trace_id, ts}`.
- `reason` returns a structured `decision_note` (one sentence, at most 25 words) with every tool call or reply. It is a summary written for display, not the model's internal reasoning.
- Events are stored in `decision_events` and served by `GET /sessions/{id}/events`, so any session can be replayed in a meeting.
- Every timeline ends with "Open full trace in Phoenix", built from the event's `trace_id` (confirm the trace URL pattern for the pinned Phoenix version).

Phoenix view

- Each serve trace carries the OpenInference `session.id` attribute set to the API `session_id`, so Phoenix's Sessions view shows the whole conversation with one trace per turn.
- Demo walkthrough: Phoenix Cloud, project `serve`, Sessions, open the session, then follow one `reason` to `guard_pre_tool` to `approval_gate` to `execute_tools` chain, pointing at the guard events and the tool result.
- Phoenix Cloud is a single-developer account, so show it from the presenter's screen and never share the login.

5. Guardrails and human sign-off

Every WRITE tool call passes deterministic guards and then a sign-off; a block or rejection returns a reason to `reason` (max 2 retries per turn) instead of failing silently.

5.1 Guardrail catalogue

Layer	Rule	Action on fail	File
Input	Prompt-injection and role-override patterns in customer turns	Strip, flag span, continue	`nodes/guard_input.py`
Input	PII (cards, phone, email) masked before tracing	Mask in spans only	`nodes/guard_input.py`
Pre-tool	Tool exists and arguments validate against JSON schema	Feedback to `reason`	`nodes/guard_pre_tool.py`
Pre-tool	Same tool with same args 3 times, or 30 steps in a turn	Force a clarifying reply or escalation	`nodes/guard_pre_tool.py`
Pre-tool	WRITE needs an explicit customer yes to a summary whose details match the args	Feedback: ask the customer to confirm	`policy_rules/common.py`
Pre-tool	Domain rules transcribed from each `policy.md` (eligibility, status preconditions, payment rules)	Feedback with the rule cited	`policy_rules/{retail,airline,telecom}.py`
Output	No claim that an action happened without a matching successful tool result	Rewrite turn	`nodes/guard_output.py`
Output	No tool call and customer-facing text in the same turn	Split turn	`nodes/guard_output.py`
Model	Served provider matches the pinned OpenRouter provider	Fail the sim, log `provider_mismatch`	`nodes/reason.py`

Layer	Rule	Action on fail	File
Pre-tool	Tool belongs to the session's chosen domain (<domain>.* only)	Block, feedback to reason	`nodes/guard_pre_tool.py`

Every guard decision is a span event with `guard.name`, `guard.verdict` and `guard.reason`, so each guard's block rate can be compared with task outcome.

5.2 Sign-off in serve mode

1. `approval_gate` calls `interrupt({tool, args, summary, policy_checks, customer_confirmation})`.
2. The graph state is persisted by `PostgresSaver`; an `approvals` row is written with status `pending`.
3. The operator reviews in the console and posts a decision to `POST /approvals/{id}`.
4. The API resumes with `Command(resume=payload)`: approve runs the tool, edit runs it with `edited_args`, reject returns a tool message with the reason.
5. Every decision is stored with reviewer, timestamp and reason for audit.

5.3 Sign-off in bench mode

No human is present, so the reviewer is a config flag and each setting is reported separately, never mixed into one number.

`approval_mode`	Behaviour	Purpose
`auto_approve`	Approves everything; logs what `rule_check` would have blocked	Raw capability and "would-have-blocked" count
`rule_check`	Deterministic policy reviewer	Headline configuration
`llm_reviewer`	Second model judges the call against the policy text	Comparison against rules

5.4 Escalation

The agent escalates through the domain's transfer tool when policy requires it or after repeated blocks. Each escalation is classed as correct (task expects a transfer), unnecessary or missed, and shown in the console queue in serve mode.

6. Models and OpenRouter configuration

All model calls go through LiteLLM to OpenRouter with one pinned provider per model, so the agent, the tau2 baseline and the user simulator make identical, reproducible requests. Setting `order` disables OpenRouter's load balancing, and `require_parameters` routes only to providers that honour every request parameter (OpenRouter provider routing).

Role	Env var	Rule
Agent (headline)	`AGENT_MODEL`	Tool-calling model; all reported scores use it
Agent (dev loop)	`DEV_MODEL`	Cheaper model for P1 to P3 iteration; never reported
User simulator	`USER_SIM_MODEL`	Fixed for every run; changing it invalidates prior comparisons

```
# src/agent/llm.py
OPENROUTER_PROVIDER = {
    "order": [os.environ["OPENROUTER_PROVIDER_ORDER"]],  # exact slug from
the model page
    "allow_fallbacks": False,
    "require_parameters": True,
}
LLM_ARGS = {"temperature": 0.0, "extra_body": {"provider":
OPENROUTER_PROVIDER}}
llm = ChatLiteLLM(model=os.environ["AGENT_MODEL"], **LLM_ARGS)  # langchain-
litellm
```

- Model strings use the LiteLLM form `openrouter/<org>/<model>` with `OPENROUTER_API_KEY` set.
- The same `LLM_ARGS` pass to tau2 through its agent and user llm-args flags (confirm names with `tau2 run --help`).
- Logged per task: tokens, cost (from LiteLLM usage), served provider, 429 and error rates.
- Bench run concurrency starts low and rises until 429s appear, then steps back one level.

7. Evaluation methodology

Every configuration runs 4 trials per task on all three domains, which yields success rate, pass^1 to pass^4, tool accuracy and escalation rate from the same simulations. The built-in tau2 `llm_agent` on the same model is always the baseline, so any delta belongs to the

architecture.

7.1 Metric definitions

Metric	Definition	Source
Task success rate	Mean tau2 reward over all simulations (equals pass^1)	tau2 results JSON
pass^k	Mean over tasks of C(c, k) / C(n, k), where n = trials and c = successes	`src/evals/metrics.py`
Action recall	Expected `evaluation_criteria.actions` matched by name and compared args	tau2 reward info plus trajectory
Write precision	WRITE calls that appear in the expected action set, over all WRITE calls	Trajectory
Invalid call rate	Calls rejected by schema or returning a tool error, over all calls	Trajectory and spans
Escalation rate	Simulations calling the transfer tool, split into correct, unnecessary and missed	Trajectory plus task criteria
Guard and approval rates	Blocks and rejections per guard, and their task outcomes	Phoenix span events
Cost and effort	Tokens, USD and steps per task	LiteLLM usage

```
\text{pass}^k = \frac{1}{|T|} \sum_{t \in T} \frac{\binom{c_t}{k}}{\binom{n}{k}}
```

Telecom scores on environment assertions as well as actions, so its tool-accuracy numbers are computed and reported separately from airline and retail. Read `docs/evaluation.md` in the pinned tau2 version before writing `metrics.py`.

7.2 Run matrix

Config	Agent	`approval_mode`	Guards	Purpose
`baseline`	tau2 `llm_agent`	built-in	none	Reference on the same model
`graph-noguard`	`langgraph_react`	`auto_approve`	off	Isolates the graph itself

Config	Agent	`approval_mode`	Guards	Purpose
`graph-full`	`langgraph_react`	`rule_check`	all	Headline result
`graph-llmrev`	`langgraph_react`	`llm_reviewer`	all	Optional reviewer comparison

- Dev loop: `--num-tasks 10 --num-trials 1` on `DEV_MODEL`.
- Phase gates: full `base` split, 4 trials, `AGENT_MODEL`, all three domains.
- Ablations in P3: switch off one guard at a time from `graph-full` and record the change in pass^1 and write precision.
- Results are committed to the repo as `results/<run_id>/<domain>/<config>.json` plus `summary.json`, where `run_id` = date + git SHA.

8. Observability with Phoenix

One trace per tau2 simulation or serve session, with eval scores written back onto the root span, so a failed pass^4 task opens straight into its trajectory. Phoenix runs in two places: self-hosted Docker for benchmark traces (no caps) and Phoenix Cloud's free tier for the Vercel demo.

Deployment

- Benchmark traces: Docker Compose on the laptop (`arizephoenix/phoenix:<pinned>` plus Postgres 14 or later via `PHOENIX_SQL_DATABASE_URL`, persistent volume). Full-trace benchmark runs happen here (Phoenix Docker docs).
- Scheduled GitHub Actions runs are metrics-only (tracing off), because the runner is discarded after the job and the volume would exceed Phoenix Cloud's free tier.
- Demo traces: Phoenix Cloud free tier at `https://app.phoenix.arize.com`, API key stored in Vercel environment variables. It is a single-developer account and may throttle ingestion on the free tier.
- Serverless rule: Vercel may freeze a function once its response is sent, so spans are exported immediately (`batch=False`) or flushed with `tracer_provider.force_flush()` before the handler returns.

Instrumentation (`src/observability/tracing.py`)

```
from phoenix.otel import register

def init_tracing(project: str):
    # Reads PHOENIX_COLLECTOR_ENDPOINT and PHOENIX_API_KEY from the
environment:
    # local Docker for benchmark runs, https://app.phoenix.arize.com on
Vercel
    return register(
        project_name=project,  # "bench-<domain>-<config>" or "serve"
        auto_instrument=True,  # openinference langchain + litellm
        batch=False,           # export each span at once; serverless may
freeze after the response
    )
```

Span attributes and events

Level	Attributes
Root span	`tau2.domain`, `tau2.task_id`, `tau2.trial`, `config.name`, `git.sha`, `model`, `provider`
Node spans	LangGraph node name, step count, retries
Events	`guard.*` decisions, `approval.*` decisions, `escalation.class`
Annotations (post-run)	reward, action recall, write precision, escalation class, pass^k membership
Session grouping (serve)	`session.id` = API session_id, so Phoenix Sessions shows the whole conversation

`src/evals/report.py` writes the annotations with `arize-phoenix-client` (confirm the method name for the pinned version). In P5 the tau2 task list is uploaded as a Phoenix dataset, so each config run becomes an experiment that can be compared in the UI.

9. Deployment on Vercel Hobby

The serve path runs as two Python functions on Vercel Hobby, backed by free Postgres; benchmarks run on a laptop or GitHub Actions, never on Vercel. Hobby is for personal, non-commercial use only, so the project moves to Vercel Pro before any client showing (Vercel Hobby plan). With Fluid compute, Hobby functions default to and max out at 300 seconds per request (Vercel changelog).

Component	Runs on	Key settings
`agent-api`	Vercel Python function (FastAPI)	`maxDuration` 300 s, SSE per turn, pooled Postgres URL
`mcp-tools`	Vercel Python function (FastMCP, streamable HTTP)	Stateless; session DB state in Postgres
Postgres	Supabase or Neon free tier	Pooled URL for functions, direct URL for migrations
Benchmark runner	Laptop (full traces) or GitHub Actions (scheduled, metrics only)	Commits `results/<run_id>/` to the repo
Phoenix	Docker for benchmarks, Phoenix Cloud free for the demo	See section 8

Serverless rules the code must follow

- One request runs the graph to the next interrupt or reply; nothing waits in memory for an approval. `POST /approvals/{id}` resumes from the Postgres checkpoint in a new request.
- `PostgresSaver` connects over the pooled URL with `{"autocommit": True, "prepare_threshold": 0}`; `checkpointer.setup()` runs once from `scripts/migrate.py`, never per request.
- Serve functions import only `src/agent`, `src/api`, `src/mcp_server` and the three domain toolkits and data files. The tau2 harness, `src/evals` and dev dependencies stay out via a separate `requirements-vercel.txt`; check the current Vercel function size limit on the first deploy.
- No background work after the response; traces are flushed before returning (section 8).
- Free Postgres tiers pause or cold-start when idle; wake the database before a demo.

Build and release

- Vercel Git integration: every push gets a preview URL; `main` is production. `vercel.json` sets functions, `maxDuration` and routes for `/api/*` and `/mcp`.
- Secrets live in Vercel environment variables: `OPENROUTER_API_KEY`, `DATABASE_URL` (pooled), `PHOENIX_COLLECTOR_ENDPOINT`, `PHOENIX_API_KEY`, model and provider variables.

- `.github/workflows/bench.yml` runs on a weekly cron, one job per domain to stay inside the 6-hour job limit, uses `OPENROUTER_API_KEY` from GitHub secrets, and commits `results/`, which redeploys `/evals`.
- Local dev mirrors production with `docker-compose.yml` (API, MCP, Postgres, Phoenix).

10. Repository layout and configuration

One repo, one `pyproject.toml`, with tau2-bench pulled in as a pinned fork so the custom agent can be registered.

```
react-agent/
  pyproject.toml                        # full dev deps: pinned tau2-bench
fork, langgraph, mcp, langchain-litellm, arize-phoenix-otel
  requirements-vercel.txt               # serve-only deps for the Vercel
functions (no tau2 harness)
  vercel.json                           # functions, maxDuration 300, routes
for /api/* and /mcp
  .env.example
  api/index.py                          # Vercel entry: exposes the FastAPI
app from src/api/main.py
  api/mcp.py                            # Vercel entry: FastMCP streamable
HTTP app
  src/agent/graph.py                    # build_graph(mode, approval_mode)
  src/agent/state.py
  src/agent/llm.py                      # OpenRouter provider pinning
  src/agent/prompts/system.md

src/agent/nodes/{guard_input,reason,guard_pre_tool,approval,execute,emit,guar
d_output}.py
  src/agent/policy_rules/{common,retail,airline,telecom}.py
  src/tau2_adapter/{agent,convert,factory}.py
  src/mcp_server/{server,session_store}.py
  src/api/main.py
  src/api/routes/{sessions,approvals,evals}.py
  src/api/ui/                           # approvals console + /evals page
  src/evals/{metrics,run_matrix,report}.py
  src/observability/tracing.py
  scripts/migrate.py                    # one-off: PostgresSaver.setup() + app
tables
  results/<run_id>/                     # committed benchmark outputs read by
/evals

demo/scenarios/{retail_exchange,airline_cancel_blocked,telecom_troubleshoot}.
yaml
  docker-compose.yml                    # local: api, mcp, postgres, phoenix
```

```
  .github/workflows/{ci,bench}.yml

tests/{test_convert,test_tool_parity,test_guards,test_policy_rules,test_metri
cs,test_approvals}.py
```

Environment (`.env.example`)

```
OPENROUTER_API_KEY=
AGENT_MODEL=openrouter/<org>/<model>
DEV_MODEL=openrouter/<org>/<model>
USER_SIM_MODEL=openrouter/<org>/<model>
OPENROUTER_PROVIDER_ORDER=<provider-slug>
APPROVAL_MODE=rule_check
MAX_STEPS_PER_TURN=30
MAX_RETRIES_PER_TURN=2
DATABASE_URL=postgresql://...          # pooled URL (Supabase or Neon) used
by functions
DATABASE_URL_DIRECT=postgresql://...   # direct URL used only by
scripts/migrate.py
PHOENIX_COLLECTOR_ENDPOINT=http://localhost:6006   #
https://app.phoenix.arize.com on Vercel
PHOENIX_API_KEY=                                   # Phoenix Cloud key on
Vercel
PHOENIX_SQL_DATABASE_URL=postgresql://.../phoenix  # local Docker Phoenix
only
RESULTS_DIR=results
OPERATOR_TOKEN=                        # shared secret for console actions
(start session, approvals)
```

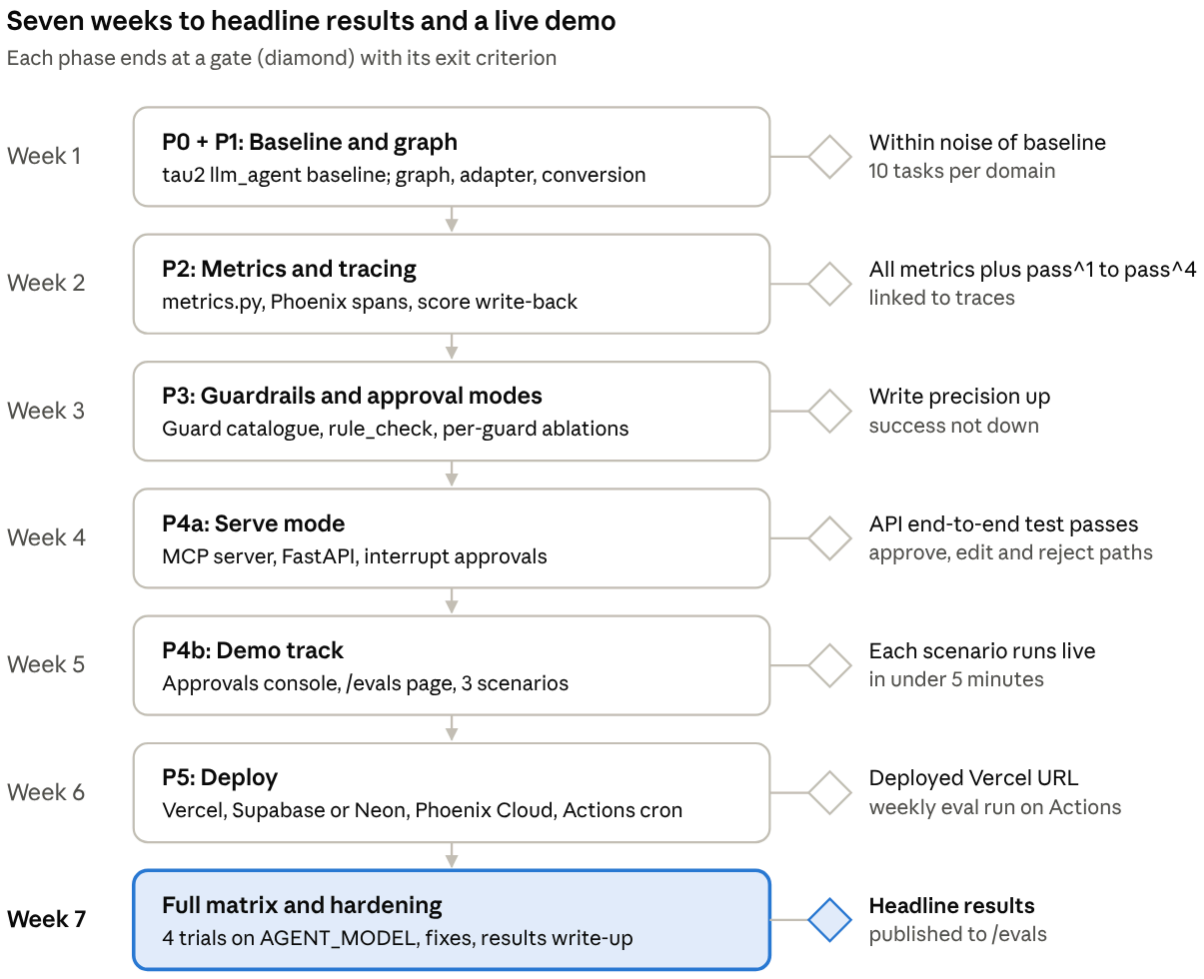
Data contracts

Table or object	Key fields
`approvals` (Postgres)	`id`, `session_id`, `thread_id`, `tool`, `args`, `edited_args`, `status`, `reviewer`, `reason`, `created_at`, `decided_at`
`session_state` (Postgres)	`session_id`, `domain`, `db_snapshot` (JSONB), `updated_at`
LangGraph checkpoints	Managed by `PostgresSaver` tables

Table or object	Key fields
`summary.json` (repo, results/<run_id>/)	`run_id`, `config`, `domain`, `success_rate`, `pass_k[1..4]`, `action_recall`, `write_precision`, `invalid_call_rate`, `escalation{correct,unnecessary,missed}`, `cost_usd`
`decision_events` (Postgres)	`id, session_id, seq, type, node, title, detail, verdict, trace_id, created_at`

11. Build plan

Seven weeks reach headline benchmark results and a deployed demo. No phase starts until the previous gate passes.



build roadmap · 7 weeks, 7 gates

Benchmark work leads (weeks 1 to 3) so the demo is built on a graph whose numbers are already known.

Tasks by phase

Week 1: P0 + P1

- [ ] Fork tau2-bench at a release tag; `uv sync`; run `llm_agent` on 10 tasks per domain
- [ ] Write `convert.py` with round-trip tests
- [ ] Build `graph.py` (bench mode) and the `LangGraphReActAgent` adapter; register `langgraph_react`
- [ ] Pin OpenRouter providers in `llm.py`

Week 2: P2

- [ ] Implement `metrics.py`: success, pass^k, action recall, write precision, invalid calls, escalation classes
- [ ] Add `tracing.py`, Docker Compose Phoenix, root span attributes
- [ ] Write `report.py` to annotate spans and emit `summary.json`

Week 3: P3

- [ ] Implement every guard in section 5.1 with fixtures from real trajectories
- [ ] Implement `auto_approve`, `rule_check`, `llm_reviewer`
- [ ] Run per-guard ablations and record the table

Week 4: P4a

- [ ] Build the FastMCP server and Postgres `session_store`; pass the parity test
- [ ] Add serve mode: `execute_tools`, `interrupt()`, `PostgresSaver`
- [ ] Implement the FastAPI routes in section 4.4 and the approvals table
- [ ] Add `GET /domains` and domain-locked sessions (domain required on `POST /sessions`, enforced in the guard and MCP server)
- [ ] Emit SSE `step` events with decision notes, persist `decision_events`, add `GET /sessions/{id}/events`, and set `session.id` on traces

Week 5: P4b

- [ ] Build the approvals console and `/evals` page in EMB brand
- [ ] Script the 3 demo scenarios in `demo/scenarios/` and rehearse
- [ ] Add the operator's domain selector and "Start session" to the console, a domain filter on the queue, and the customer chat page with a read-only domain badge
- [ ] Build the Decision timeline panel (live and replay) with the "Open full trace in Phoenix" link, and add the Phoenix walkthrough to each demo scenario

Week 6: P5

- [ ] Create the Supabase or Neon project; run `scripts/migrate.py` against the direct URL
- [ ] Add `vercel.json`, `api/index.py`, `api/mcp.py` and `requirements-vercel.txt`; deploy `agent-api` and `mcp-tools` to Vercel Hobby and check bundle size
- [ ] Set Vercel environment variables and point demo tracing at Phoenix Cloud; confirm spans arrive after each request
- [ ] Add `.github/workflows/bench.yml` with a weekly cron, one job per domain, committing `results/`

Week 7: full matrix and hardening

- [ ] Run the full matrix with 4 trials on `AGENT_MODEL`
- [ ] Fix top failure clusters from Phoenix; rerun affected domains
- [ ] Write the results summary and publish to `/evals`

12. Testing strategy

Unit tests guard the conversions and rules that fail silently; the tau2 runs themselves are the integration test, gated per phase.

Level	What	Gate
Unit	`convert.py` round-trips tau2 and LangChain messages, including tool-call IDs and multi-tool messages	Every PR
Unit	Each guard and policy rule with passing and failing fixtures taken from real trajectories	Every PR
Unit	`pass_hat_k` against hand-computed cases; `n < k` raises	Every PR
Contract	MCP tool schemas equal `env.get_tools()` for all three domains	Every PR
Integration	Serve mode: interrupt, approve, edit and reject paths resume correctly from Postgres	Every PR (Docker Compose)
Smoke eval	10 tasks per domain, 1 trial, `DEV_MODEL`	Every merge to `main`
Full eval	Full run matrix, 4 trials, `AGENT_MODEL`	Phase gates and nightly
Demo rehearsal	All 3 scenarios end to end on the deployed URL	Before any client showing

Level	What	Gate
Deploy check	Vercel preview builds, bundle within limit, /healthz responds, one chat turn completes under 300 s	Every PR (Vercel preview)
Unit	A session binds only its chosen domain's tools and policy; session creation without an operator token, or with a missing or unknown domain, is rejected; cross-domain tool calls are refused by the guard and the MCP server	Every PR
Integration	Every graph step emits one step event with a decision note and trace id; replay from decision_events matches the live stream; the Phoenix link opens the right trace	Every PR

13. Risks, mitigations and open decisions

The largest risks are noise in the scores (provider routing, the user simulator) and guards that block correct actions; both are measured rather than assumed.

Risk	Impact	Mitigation
OpenRouter routes the same model to different providers	pass^k variance unrelated to the agent	Pin provider, `allow_fallbacks: false`, fail sims on `provider_mismatch`
User simulator noise	Unstable success rates	Fixed `USER_SIM_MODEL`, 4 trials, compare configs on identical settings
Full matrix cost	Budget overrun	Dev loop on `DEV_MODEL` with 10 tasks; full matrix only at gates; log USD per task
Over-strict guards	Lower success despite better safety	Per-guard ablation in P3; track block rate against task outcome
Message conversion bugs	Silent random failures	Round-trip unit tests before P1 exit
Bench rejections change trajectories	Misleading comparisons	Report each `approval_mode` separately, never mixed

Risk	Impact	Mitigation
Telecom dual control	Confirmation guard blocks guidance turns	Domain-specific rules; telecom metrics reported separately
Session state across serverless invocations	Lost or mixed DB state	State in Postgres keyed by `session_id`
tau2 upstream changes	Scores not comparable	Pin the fork to a release tag; re-score old runs only with `--fresh-tasks` deliberately
Vercel Hobby licence is non-commercial	Client demo breaches the plan terms	Personal build on Hobby; switch to Pro before any client showing
300 s per-request cap on Hobby	Long agent turns time out (504)	One request per graph step to the next interrupt or reply; step cap of 30; log turn duration
Function bundle too large	Deploy fails	Serve-only requirements file; tau2 harness and eval code excluded; size checked on first preview deploy
Free Postgres pauses or cold-starts	Slow or failed first request in a demo	Wake the database before demos; pooled URL; retry once on connect
Phoenix Cloud free tier throttles	Missing demo traces	Demo traffic only goes to Phoenix Cloud; benchmark traces stay on local Docker Phoenix

Open decisions

- [ ] Choose `AGENT_MODEL`, `DEV_MODEL` and `USER_SIM_MODEL` on OpenRouter, with the pinned provider for each.
- [ ] Decide whether the user simulator matches the public leaderboard setup for external comparability.
- [ ] Before any client showing: move the Vercel project to Pro and confirm EMB branding on the console.